End-to-End LLM Chatbot with RAG, Monitoring & Deployment

---

1. Project Overview

This project involves designing and deploying a conversational chatbot powered by a Large Language Model (LLM), enhanced with Retrieval-Augmented Generation (RAG) so it can answer questions grounded in a specific knowledge base rather than relying purely on the model's pretrained knowledge. The system is then wrapped with monitoring to track usage, latency, and answer quality, and finally deployed as a usable web application.
This project reflects the modern, real-world workflow for building production-grade LLM applications — going beyond simple prompt-response demos into retrieval, evaluation, observability, and deployment.

---

2. Problem Statement

Plain LLMs are powerful but limited: they can hallucinate facts, lack access to private/domain-specific data, and provide no visibility into how well they're performing once deployed. Organizations need chatbots that can answer questions using their own documents (manuals, FAQs, policies, notes) accurately, while also giving teams insight into system health and answer quality over time.
This project solves that by building a chatbot that:

1. Retrieves relevant context from a custom document set before generating an answer.
2. Logs and monitors interactions to catch poor answers, slow responses, or unusual usage.
3. Runs as a deployed, accessible application rather than a local script.

---

3. Objectives

- Build a document ingestion pipeline that chunks and embeds a custom knowledge base.
- Implement a vector store for semantic search/retrieval.
- Integrate an LLM (open-source or API-based) with the retrieved context to generate grounded answers (RAG pipeline).
- Build a simple chat interface for users to interact with the bot.
- Add monitoring for latency, token usage, retrieval quality, and user feedback (thumbs up/down).
- Deploy the chatbot as a live, accessible web application.

---

4. Scope

In Scope:

- Document preprocessing (PDF/text/FAQ ingestion, chunking)
- Embedding generation and vector database setup
- RAG pipeline: retrieval + prompt construction + LLM response generation
- Basic chat UI (web-based)
- Monitoring dashboard: request logs, latency, token cost, feedback tracking
- Deployment (cloud or containerized local demo)

Out of Scope (unless extended):

- Fine-tuning the LLM itself
- Multi-turn long-term memory across sessions
- Enterprise-grade authentication/security
- Multi-language support

---

5. System Architecture

```
User Query
    ↓
Chat Interface (Web UI)
    ↓
RAG Pipeline:
  1. Embed user query
  2. Retrieve top-k relevant chunks (Vector DB)
  3. Construct prompt (query + retrieved context)
  4. Send to LLM → Generate answer
    ↓
Response returned to user
    ↓
Logging Layer (query, retrieved docs, response, latency, feedback)
    ↓
Monitoring Dashboard (usage, latency, quality metrics, drift/failure alerts)
    ↓
Deployment (Dockerized app on cloud/local server)
```

---

6. Suggested Tech Stack

Layer	Tools/Technologies
Language	Python
LLM	OpenAI API / Llama / Mistral (via API or local inference)
RAG Framework	LangChain or LlamaIndex

Embeddings	OpenAI Embeddings, Sentence-Transformers
Vector Store	FAISS, Chroma, or Pinecone
Backend/API	FastAPI / Flask
Frontend/Chat UI	Streamlit / React / simple HTML-JS chat widget
Monitoring	Prometheus + Grafana, or custom logging dashboard (Streamlit/Plotly)
Deployment	Docker, cloud platform (Render, AWS, GCP, or Hugging Face Spaces)

---

7. Knowledge Base / Dataset

Any domain-specific document set, such as:

- Company FAQ / policy documents
- Course notes or textbook chapters
- Product manuals
- A public dataset like Wikipedia articles on a chosen topic

---

8. Evaluation Metrics

- Retrieval quality: relevance of retrieved chunks (manual or automated scoring)
- Answer quality: faithfulness/groundedness to retrieved context, hallucination rate
- System performance: response latency, token usage/cost
- User feedback: thumbs up/down ratio, follow-up question rate

---

9. Monitoring Dashboard Features

- Usage Metrics: number of queries, active sessions, peak usage times
- Performance Metrics: end-to-end latency, retrieval time vs. generation time
- Quality Metrics: feedback scores, flagged/failed responses, hallucination flags
- Cost Tracking: token consumption and estimated API cost over time

---

10. Learning Outcomes

By completing this project, the student will demonstrate:

- Practical understanding of how RAG improves LLM reliability and grounding
- Experience building with vector databases and embedding models
- Ability to integrate an LLM into a working application via API/backend
- Familiarity with LLMOps practices: logging, monitoring, and evaluation of generative systems
- End-to-end experience: data → retrieval → generation → deployment

---

11. Deliverables

- Source code repository (ingestion pipeline, RAG backend, chat UI)
- Vector database populated with the chosen knowledge base
- Deployed, accessible chatbot application (link or local demo instructions)
- Monitoring dashboard showing live/historical metrics
- Project report covering architecture, evaluation results, and key learnings